NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
MIDTERM ASSESSMENT FOR
Semester 2 AY2022/2023

CS2030S Programming Methodology II

April 2023 Time Allowed 120 minutes

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains 19 questions and comprises 14 printed pages, including this page.
2. Write all your answers in the answer sheet provided.
3. The total marks for this assessment is 100. Answer **ALL** questions.
4. This is an **OPEN BOOK** assessment. You are only allowed to refer to hard-copy materials.
5. All programs in this assessment paper use Java 17 and are compiled with the flags `-Xlint:unchecked` and `-Xlint:rawtypes`.
6. Import statements for all programs are omitted. You may assume that the necessary classes are imported.
7. An abridged API for Java `Stream` can be found on Page 14.

Question	Points
1 - 11	33
12	4
13	7
14	8
15	12
16	9
17	6
18	15
19	6
TOTAL	100

Section I: Multiple Choice Questions

Pick the most appropriate answer and shade the corresponding letter on the answer sheet. If none of the answers is appropriate, shade X in the corresponding answer box on the answer sheet. If more than one answer is equally appropriate, shade ONLY one of the answers in the answer box on the answer sheet.

Each question is worth 3 marks.

1. The exception `ClassCastException` is thrown during run time when the code tries to cast an object to an incompatible type. Which of the following concepts in CS2030S, when used properly, helps to reduce the chances of `ClassCastException` being thrown?

   A. The Liskov Substitution Principle
   B. Generics
   C. Raw types
   D. Pure functions
   E. Immutability

   Shade X in the answer box if none of the above is correct.

2. The `NullPointerException` is thrown when the code tries to use a member of a `null` object. Which of the following concepts in CS2030S helps to reduce the chances of `NullPointerException` being thrown during run time?

   A. Dynamic binding
   B. Type inference
   C. The `Maybe` Monad
   D. Lazy evaluation
   E. Pokémon exceptions

   Shade X in the answer box if none of the above is correct.

3. Writing code with control flow and logic that is easy to reason about leads to code that is easier to debug and maintain. Which of the following concepts in CS2030S does NOT help to make the code easier to reason about?

   A. The Liskov Substitution Principle
   B. Referential transparency
   C. Immutable classes
   D. Pure functions
   E. Aliasing

   Shade X in the answer box if none of the above is correct.

4. Software evolves over time. Changes in one part of the software can impact other parts. Which of the following concepts in CS2030S does NOT help in limiting the impact of code changes as software evolves?

   A. The Liskov Substitution Principle
   B. Information hiding
   C. Tell, don't ask
   D. Method overloading
   E. Pure function

   Shade X in the answer box if none of the above is correct.

5. Which of the following is a trait of Java as a strongly-typed programming language?

   A. It allows type casting with strict rules.
   B. It allows arrays to be covariant.
   C. It performs type inferencing.
   D. It allows type erasure.
   E. It allows raw types.

   Shade X in the answer box if none of the above is correct.

6. Consider the following program:

```
class Z {
  void foo() {
  }

  static void bar(Z z, int i) {
    z.foo(); // Line A
  }

  public static void main(String[] args) {
    Z.bar(new Z(), 3); // Line B
  }
}
```

   Which of the following can be determined by the Java compiler when the program above is compiled?

   A. The value of parameter `i` at Line A.
   B. The run-time type of parameter `z` at Line A
   C. Whether Line A will ever be reached during execution.
   D. Which implementation of `foo` is invoked at Line A.
   E. Which implementation of `bar` is invoked at Line B.

   Shade X in the answer box if none of the above is correct.

7. Consider the following classes:

```
class Point {
   :
}

class Rectangle {
  private Point topLeft;
  private Point lowerRight;
   :
}
```

   Details of the two classes `Point` and `Rectangle` are omitted. The relationship between the classes `Point` and `Rectangle` is called:

   A. inheritance
   B. covariance
   C. composition
   D. subtyping
   E. nesting class

   Shade X in the answer box if none of the above is correct.

8. Consider the `Box` class below:

```
class Box<T> {
    :
  public static <T> Box<T> of(T value) {
    :
  }

  public static <T> void foo(Box<? super T> box) {
    :
  }
}
```

Additionally, we have the following types:

```
interface I { }
class A { }
class AI extends A implements I { }
class AI1 extends AI { }
```

and variables

```
Object o;
I i;
A a;
AI ai;
AI1 ai1;
```

Which of the following method invocations would lead to a type mismatch and thus a compilation error?

A. `Box.foo(Box.of(o));`
B. `Box.foo(Box.of(i));`
C. `Box.foo(Box.of(a));`
D. `Box.foo(Box.of(ai));`
E. `Box.foo(Box.of(ai1));`

Shade X in the answer box if none of the above is correct.

9. Consider the following program:

```
class Main {
  public static void main(String[] args) {
    Producer<Box<Integer>> p = () -> {
      Box<Integer> b = Box.of(5);
      // Line C
      return b;
    };

    p.produce();
  }
}
```

Which of the following statements is correct about the state of the stack when the program executes and reaches Line C (following CS2030S convention)? Here, the stack refers to the stack in the stack and heap diagram.

A. The stack is empty.
B. Only the stack frame for `main` is on the stack.
C. Only the stack frames for `main` and `produce` are on the stack.
D. The stack frames for `main`, `produce`, and `of` are on the stack.
E. This is a trick question as Line C is never called.

Shade X in the answer box if none of the above is correct.

10. Consider the two interfaces and one class below.

```
interface Func1<T1, R> {
  R call(T1 t1);
}

interface Func2<T1, T2, R> {
  R call(T1 t1, T2 t2);
}

class A {
  public A foo1(A x) {
    return null;
  }

  public A foo2(A x, A y) {
    return null;
  }
}
```

Additionally, you are given the following variables.

```
A a = new A();
Func1<A, A> f1;
Func2<A, A, A> f2;
```

Which of the following method references CANNOT be assigned into either `f1` or `f2`?

A. `A::foo1`
B. `A::foo2`
C. `a::foo1`
D. `a::foo2`

Shade X in the answer box if none of the above is correct.

11. Consider the following program:

```
class CF {
   :
  private static void run(int x) {
    doSomething();
    System.out.print(x);
  }

  public static void main(String[] args) {
    CompletableFuture<Void> cf = CompletableFuture.runAsync(() -> run(4));
    cf.thenRunAsync(() -> run(3));
    cf.thenRunAsync(() -> run(2)).join();
    cf.thenRunAsync(() -> run(1)).join();
  }
}
```

The method `doSomething` takes a non-deterministic amount of time each time it is called. Which of the following statement is NOT correct about the possible printed values?

A. If 1 is printed, it is always printed after 4.
B. If 2 is printed, it is always printed after 4.
C. If 3 is printed, it is always printed after 4.
D. If 3 is printed, it is always printed before 2 and 1.
E. 1, 2, and 4 are always printed (but not necessary in this order).

Shade X in the answer box if none of the above is correct.

Section II: Short Questions

12. (4 points) Consider the following classes:

```
class E1 extends Exception { }
class E2 extends E1 { }

class W {
  static void m1() throws E1 {
    m2();
    // Line D
    m2();
    // Line E
  }

  static void m2() throws E1 {
    m3(0);
    // Line F
    m3(1);
    // Line G
  }

  static void m3(int i) throws E1 {
    if (i == 0) {
      throw new E2();
    }
    // Line H
  }

  public static void main(String[] args) {
    try {
      m1();
    } catch (E2 e) {
      // Line J
    } catch (E1 e) {
      // Line K
    } finally {
      // Line L
    }
  }
}
```

Trace through the execution flow of the program when the program is executed, and write down the lines that would be reached during the execution in order.

For example, if the execution reaches Lines D, E, F, and G in that order, write DEFG on the answer sheet.

13. (7 points) What does each of the following stream pipelines print?

(a) (3 points)

```
Stream.of(1, 2, 3, 4)
  .map(n -> Stream.iterate(1, i -> i + 1)
     .limit(n)
     .map(x -> x.toString())
     .reduce("", (x, y) -> x + y))
  .forEach(x -> System.out.print(x + " "));
```

(b) (4 points)

```
Stream.of(1, 2, 3)
  .flatMap(x -> Stream.of(2, 3, 4)
      .map(y -> x * y)
      .filter(z -> z % 2 == 0))
  .forEach(x -> System.out.print(x + " "));
```

14. (8 points) A store has a program for calculating the discount prices on all products. For their big sale, there is a requirement that for products above $100, there is a discount of 10%.

Consider the class `SaleDiscount`:

```
class SaleDiscount {
  private double price;
  private int productID;

  public SaleDiscount(double price, int productID) {
    this.price = price;
    this.productID = productID;
  }

  public double getPrice() {
    return this.price;
  }

  public void setPrice(double price) {
    this.price = price;
  }

  public void discount() {
    if (price > 100) {
      this.price = 0.90 * this.price;
    } // Line M
  }
}
```

Sales are not going well, so the store owners decide to also discount 50% off of all products but only for products costing $100 or lower. Consider the class `FireSaleDiscount`:

```
class FireSaleDiscount extends SaleDiscount {
  public FireSaleDiscount(double price, int productName) {
    super(price, productName);
  }

  @Override
  public void discount() {
    double price = this.getPrice();
    double discount = (price > 100) ? 0.90 : 0.5;
    this.setPrice(discount * price);
  }
}
```

(a) (3 points) Does the class `FireSaleDiscount` violate the Liskov Substitution Principle? If so, why? If not, why not?

(b) (5 points) Consider the following class `Store`

```
class Store {
  public static void main(String[] args) {
    int productID = 2030;

    SaleDiscount jPhone = new SaleDiscount(1200.0, productID);
    SaleDiscount jPhonePlus = new SaleDiscount(1700.0, productID);

    jPhone.discount();
  }
}
```

Draw the stack and heap diagram, when the program reaches Line M. Line M is described in the `SaleDiscount` class. Label your stack frames, objects, and variables clearly. You may omit `args` in your diagram.

15. (12 points) Consider the following Java classes.

```
class A {
  public void f(B param) { }
  public void f(C param) { }
}

class B extends A {
  public void f(A param) { }
  public void f(B param) { }
}

class C extends B {
  public void f(C param) { }
  public void f(B param) { }
}
```

Additionally, you are given the following variables to work with.

```
A a = new C();
B b = new B();
C c = new C();
```

For each of the questions below, if there is no answer, simply write *"no answer"*. If there are multiple possible answers, you may choose any answer.

(a) (3 points) Which of the methods above cannot be invoked using only the given variables? You are not allowed to use the `new` keyword to create a new instance. Write your answer in the following format `ClassName::method(ParameterType)`.

(b) (3 points) You are allowed to add the statement `super.f(param);` to **one and only one** of the methods, in only one of the classes. Which method should the statement `super.f(param);` be added to, such that there is no answer for Part (a) above? Write your answer in the following format: `ClassName::method(ParameterType)`.

(c) (3 points) Write a method invocation that will invoke the method `A::f(C)`. Your method invocation can only use the given variables as the target of invocation and argument.

(d) (3 points) Write a method invocation that will cause a compilation error because there is no *suitable method* that can be found. Your method invocation can only use the given variables as the target of invocation and argument.

16. (9 points) For each of the following code snippets, we will compile using the `-Xlint:unchecked` and `-Xlint:rawtypes` flags.

(a) (3 points) Consider the following class and interface:

```
class A {
}

@FunctionalInterface
interface Producer {
  int produce();
}
```

and the following snippet:

```
A a = new A();
Producer p = (Producer) a;
```

Explain why the Java compiler allows `a` to be cast into `Producer` without a compilation error and without a compilation warning, even though it would cause a run-time error during execution.

(b) (3 points) Now consider the code below. The difference from Part (a) is that `Producer` is now a generic interface.

```
class A {
}

@FunctionalInterface
interface Producer<T> {
  T produce();
}
```

and the following snippet:

```
A a = new A();
Producer<Integer> p = (Producer<Integer>) a;
```

Explain why the Java compiler produces a warning when casting `a` into `Producer<Integer>`.

(c) (3 points) Now consider the code below. The difference from Part (a) is that class `A` is declared as `final`.

```
final class A {
}

@FunctionalInterface
interface Producer {
  int produce();
}
```

and the following snippet:

```
A a = new A();
Producer p = (Producer) a;
```

Explain why the Java compiler produces a compilation error. You may contrast this to your answer in Part (a) of this question.

17. (6 points) Recall that the `cs2030s.fp` package includes the following `Transformer` and `Combiner` functional interfaces.

```
@FunctionalInterface
interface Transformer<T, R> {
  R transform(T t);
}

@FunctionalInterface
interface Combiner<S, T, R> {
  R combine(S s, T t);
}
```

We want to create the class called `Curry` with two static methods. In the sample runs below, we have the following code fragments.

```
Combiner<Integer, Integer, Integer> f1 = (x, y) -> x - y;
Transformer<Integer, Transformer<Integer, Integer>> f2 = x -> y -> x - y;
```

Answer the question by filling in the corresponding blanks on the answer sheet.

```
class Curry {
  public static <S, T, R> __BLANK1__ curry(__BLANK2__ f) {
    return __BLANK3__;
  }
  public static <S, T, R> __BLANK4__ uncurry(__BLANK5__ f) {
    return __BLANK6__;
  }
}
```

(a) (3 points) Implement a static method `curry` that takes in a `Combiner` and returns a `Transformer` satisfying the following sample run.

```
jshell> f1.combine(2,1)
$.. ==> 1
jshell> Curry.curry(f1).transform(2).transform(1)
$.. ==> 1
```

(b) (3 points) Implement a static method `uncurry` that takes in a `Transformer` and returns a `Combiner` satisfying the following sample run.

```
jshell> f2.transform(2).transform(1)
$.. ==> 1
jshell> Curry.uncurry(f2).combine(2,1)
$.. ==> 1
```

18. (15 points) Consider the following implementation of `InfiniteList`, taken from the notes:

```
class InfiniteList<T> {
  public final Producer<T> head;
  public final Producer<InfiniteList<T>> tail;

  private InfiniteList(Producer<T> head, Producer<InfiniteList<T>> tail) {
    this.head = head;
    this.tail = tail;
  }

  public static <T> InfiniteList<T> iterate(T seed, Transformer<T, T> next) {
    return new InfiniteList<T>(
        () -> seed,
        () -> InfiniteList.iterate(next.transform(seed), next)
        );
  }

  public T head() {
    T h = this.head.produce();
    return h == null ? this.tail.produce().head() : h;
  }

  public InfiniteList<T> tail() {
    T h = this.head.produce();
    return h == null ? this.tail.produce().tail() : this.tail.produce();
  }

  public InfiniteList<T> filter(BooleanCondition<? super T> cond) {
    return new InfiniteList<T>(
        () -> cond.test(this.head()) ? this.head() : null,
        () -> this.tail().filter(cond)
        );
  }
}
```

The following functional interfaces are used:

```
@FunctionalInterface
interface Producer<T> {
  T produce();
}

@FunctionalInterface
interface BooleanCondition<T> {
  boolean test(T t);
}

@FunctionalInterface
interface Transformer<T, R> {
  R transform(T t);
}
```

(a) (4 points) Implement a method called `lazyGet` inside the `InfiniteList` class that takes in an `int` index `i` and lazily returns the i-th element from the list as a producer of the element.
The method is declared as:

```
public Producer<T> lazyGet(int i) {

}
```

and can be used as follows:

```
jshell> Transformer<Integer, Integer> f = x -> {
   ...>     System.out.println(" > " + (x + 2));
   ...>     return x + 2;
   ...> }
jshell> Producer<Integer> p = InfiniteList.iterate(0, f).lazyGet(0)
jshell> p.produce()
$.. ==> 0
jshell> Producer<Integer> p = InfiniteList.iterate(0, f).lazyGet(2)
jshell> p.produce()
 > 2
 > 4
$.. ==> 4
```

Fill in the body of the method `lazyGet`.

(b) (6 points) Write a method called `skip` inside the `InfiniteList` class with a parameter `x` of type `T`. The method skips all beginning elements that are equal to `x` and returns the remaining elements as an infinite list. Checking for equality should be done using the `equals` method.

For instance, calling `skip(0)` on the infinite list 0, 0, 0, 1, 1, 1, 2, 2, 2, . . . returns the infinite list 1, 1, 1, 2, 2, 2 . . .. Calling `skip(1)` on the infinite list 0, 0, 0, 1, 1, 1, 2, 2, 2, . . . returns 0, 0, 0, 1, 1, 1, 2, 2, 2, ...

The method `skip` has the following declaration and should be executed **eagerly**.

```
private InfiniteList<T> skip(T x) {

}
```

Fill in the body of the method `skip`.

(c) (5 points) Using the method `skip` above, write a method called `unique` inside the `InfiniteList` class that lazily returns a new infinite list where consecutive elements that are the same are removed. For instance, calling `unique()` on the infinite list 0, 0, 1, 1, 1, 0, 0, 1, 1, 2, 2, . . ., returns 0, 1, 0, 1, 2, ...

The method `unique` has the following declaration and should be executed **lazily**.

```
public InfiniteList<T> unique() {

}
```

Fill in the body of the method `unique`.

19. (6 points) Consider a monad called `Monad<X>` that adds a context to the computation on a value of type `X`. We have the following methods `em`, `fm`, `gm`, and `hm` that take in a value of type `X` and return a `Monad<X>`. We wish to apply the four methods, in order, to a value `x`, like so:

```
Monad<X> result = Monad.of(x)
  .flatMap(i -> em(i))
  .flatMap(i -> fm(i))
  .flatMap(i -> gm(i))
  .flatMap(i -> hm(i));
```

When you asked an AI to write a program snippet to chain the methods together, the following program is generated:

```
Monad<X> efm(X x) {
  return Monad.of(x)
    .flatMap(i -> em(i));
    .flatMap(i -> fm(i));
}

Monad<X> ghm(X x) {
  return Monad.of(x)
    .flatMap(i -> gm(i))
    .flatMap(i -> hm(i))
}

Monad<X> result = Monad.of(x)
  .flatMap(i -> efm(i))
  .flatMap(i -> ghm(i));
```

The AI insists that the two programs are equivalent but refuses to prove it to you. So you set out to prove it yourselves. Fill in the blanks below to show that the two expressions are equivalent:

We start with:

```
Monad.of(x)
    .flatMap(i -> efm(i))
    .flatMap(i -> ghm(i))
```

Following the AI's implementation, we have

```
Monad.of(x)
    .flatMap(i -> Monad.of(i).flatMap(i -> em(i)).flatMap(i -> fm(i)))
    .flatMap(i -> Monad.of(i).flatMap(i -> gm(i)).flatMap(i -> hm(i)))
```

Applying Monad's `__BLANK7__` law, we have

```
Monad.of(x)
    .flatMap(i -> ____BLANK8____)
    .flatMap(i -> ____BLANK9____)
```

Applying Monad's `__BLANK10__` law, we have

```
Monad.of(x)
    .flatMap(i -> em(i)).flatMap(i -> fm(i))
    .flatMap(i -> gm(i)).flatMap(i -> hm(i))
```

And so, the AI is correct and the two programs are equivalent.

Stream API (Abridged)

filter

```
Stream<T> filter(Predicate<? super T> predicate)
```

Returns a stream consisting of the elements of this stream that match the given predicate.

map

```
<R> Stream<R> map(Function<? super T,? extends R> mapper)
```

Returns a stream consisting of the results of applying the given function to the elements of this stream.

flatMap

```
<R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)
```

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is closed after its contents have been placed into this stream. (If a mapped stream is null an empty stream is used, instead.)

limit

```
Stream<T> limit(long maxSize)
```

Returns a stream consisting of the elements of this stream, truncated to be no longer than maxSize in length.

forEach

```
void forEach(Consumer<? super T> action)
```

Performs an action for each element of this stream.

reduce

```
T reduce(T identity, BinaryOperator<T> accumulator)
```

Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value.

of

```
static <T> Stream<T> of(T... values)
```

Returns a sequential ordered stream whose elements are the specified values.

iterate

```
static <T> Stream<T> iterate(T seed, UnaryOperator<T> f)
```

Returns an infinite sequential ordered Stream produced by iterative application of a function f to an initial element seed, producing a Stream consisting of seed, f(seed), f(f(seed)), etc.

END OF PAPER